

BME680 Based Air Quality Monitoring and Temperature Alarm System Using Rugged Board and ThingsBoard

Internship Project Report

Submitted By: Aditya Shetty

Johann Jose Jinu

Project Duration: 4 Weeks Embedded Systems / IoT Internship

Abstract

This internship project presents the development of a smart environmental monitoring and temperature alarm system built around the BME680 environmental sensor, a Rugged Board running Embedded Linux, and a self-hosted ThingsBoard server for cloud-connected telemetry visualization. The system continuously measures temperature, humidity, atmospheric pressure, and gas resistance, and it converts these physical measurements into useful monitoring data for real-time observation and decision making.

The project combines hardware interfacing, embedded software, network communication, and dashboard integration into a single operational prototype. The BME680 sensor communicates with the Rugged Board through the I2C interface, where an embedded application reads and processes the sensor output. The processed values are transmitted through HTTP to ThingsBoard, where they are displayed using dashboard widgets and stored for monitoring purposes.

An additional safety-oriented feature of the project is the implementation of a local temperature alarm using an SFM-27 buzzer connected to a GPIO pin of the Rugged Board. When the temperature rises above 30°C, the buzzer activates locally and an alarm signal is also sent to ThingsBoard, enabling remote alert visualization. This

combination of local and remote alarm handling makes the system more practical for industrial and laboratory use cases.

The completed work demonstrates the integration of Embedded Linux, sensor APIs, I2C communication, GPIO control, HTTP telemetry, and IoT dashboarding within a single Industrial IoT application. It also provides a solid foundation for future expansion such as MQTT integration, advanced air-quality computation, long-term data logging, and multi-node deployments.

Table of Contents

1. Introduction
2. Background and Motivation
3. Problem Statement
4. Objectives
5. Scope of the Project
6. Literature and Technology Overview
7. System Overview
8. Hardware Description
9. Software Description
10. Circuit Diagram and Wiring Details
11. Sensor Interfacing Methodology
12. Data Acquisition Flow
13. AQI Estimation Method
14. Telemetry Communication using HTTP
15. ThingsBoard Dashboard Design
16. Temperature Alarm Design
17. Working Principle of the System

- 18. Implementation Approach
- 19. Testing Procedure
- 20. Results and Observations
- 21. Advantages of the Proposed System
- 22. Challenges Faced
- 23. Skills Acquired
- 24. Limitations
- 25. Future Enhancements
- 26. Conclusion
- 27. References

1.Introduction

Environmental monitoring is an important requirement in factories, laboratories, data rooms, storage units, and enclosed workspaces where physical conditions directly affect equipment life, process quality, and human safety. Parameters such as temperature and humidity influence comfort and reliability, while air quality conditions may affect health, ventilation quality, and operational standards.

In many small and medium installations, monitoring is often done manually or through standalone devices that display values locally but do not provide centralized access, long-term tracking, or remote alarms. Such approaches are limited because abnormal conditions may go unnoticed when no operator is physically present near the device.

The rise of the Internet of Things has enabled the development of compact, network-connected sensing nodes that can collect data at the edge, process it locally, and publish it to dashboards for remote monitoring. These systems are especially useful in modern industrial environments where preventive maintenance and timely alerts reduce downtime and improve safety.

This project was developed with that practical requirement in mind. It integrates the BME680 environmental sensor with a Rugged Board running Embedded Linux and a ThingsBoard server that receives telemetry, displays widgets, and manages alarms. By

also adding a local buzzer, the project ensures that alerts are available even without dashboard access.

2. Background and Motivation

The BME680 is a compact environmental sensor that combines temperature, humidity, pressure, and gas sensing in one package. This makes it a useful choice for embedded monitoring projects where board space, power efficiency, and multi-parameter measurement are important considerations.

The Rugged Board platform offers a Linux-capable embedded environment with accessible peripheral interfaces such as I2C and GPIO. This makes it appropriate for educational and prototype IoT projects where both low-level hardware access and higher-level networking functions are required.

ThingsBoard is widely used as an open IoT platform because it provides telemetry ingestion, dashboards, alarms, rule chains, and device-centric monitoring workflows. Integrating the Rugged Board with ThingsBoard makes the project more relevant to real industrial IoT applications rather than keeping it as a simple isolated sensor demo.

The motivation behind this project was to gain hands-on experience in building a complete edge-to-dashboard monitoring solution and to understand how sensing, embedded processing, and remote visualization are connected in a real implementation.

3. Problem Statement

A practical environmental monitoring system must do more than simply read raw sensor values. It should be able to acquire data reliably, process it meaningfully, present it clearly to users, and generate alerts when conditions exceed acceptable limits.

The main problem addressed in this project is the design and development of an embedded system that can measure environmental conditions continuously and provide both local and remote alarms in response to high temperature. The solution must run on an Embedded Linux platform, interface correctly with the sensor hardware, and communicate with a remote IoT dashboard using a standard protocol.

The project therefore addresses not only sensing but also integration challenges related to communication protocols, GPIO-based actuation, telemetry formatting, and dashboard-level alarm handling.

4. Objectives

- Interface the BME680 environmental sensor with the Rugged Board using I2C communication.
- Acquire temperature, humidity, pressure, and gas resistance readings periodically.
- Estimate a relative air quality indicator using gas resistance data.
- Transmit telemetry to a self-hosted ThingsBoard server through HTTP.
- Create a real-time dashboard for environmental data visualization.
- Implement a local alarm using an SFM-27 buzzer controlled from a GPIO pin.
- Generate remote alarms in ThingsBoard when temperature crosses 30°C.
- Develop practical experience in Embedded Linux, sensor APIs, networking, and IoT system integration.

5. Scope of the Project

The scope of the project covers the design, implementation, and testing of a prototype environmental monitoring node using a BME680 sensor and a Rugged Board. It includes the sensor interface, embedded application logic, HTTP telemetry transfer, dashboard visualization, and high-temperature alarm handling.

The project is intended as a proof-of-concept suitable for internship-level demonstration and learning. While the implementation is functional and technically complete for a prototype, it is not intended to serve as a fully certified industrial instrument.

The AQI value used in the project is a relative estimation based on gas resistance and is suitable for trend observation rather than compliance-grade air quality reporting.

6. Literature and Technology Overview

Embedded environmental monitoring systems typically combine one or more sensors, a microcontroller or processor platform, a communication method, and a visualization or storage backend. Each layer must be designed carefully to ensure compatibility and stable operation.

The BME680 has been used in many indoor monitoring and smart-building applications because it provides a compact multi-sensor solution. It is especially useful when the goal is to observe general environmental quality rather than to measure particulate matter directly.

I2C remains one of the most widely used interfaces for sensors because it reduces wiring complexity and supports multiple devices on the same bus. In Linux-based systems, I2C support through the kernel and user-space tools makes debugging and integration more manageable.

For IoT communication, both HTTP and MQTT are commonly used. HTTP is straightforward to implement and test, which makes it a practical choice for early prototypes. Dashboards such as ThingsBoard add value by converting raw telemetry into visual insights and actionable alarms.

7. System Overview

The complete system consists of sensing, processing, communication, local alerting, and remote monitoring layers. The BME680 acts as the sensing layer, the Rugged Board performs local processing and control, the network connection carries telemetry to the server, and ThingsBoard forms the cloud-style monitoring layer.

The sensed values are periodically acquired and packaged into telemetry fields. These fields are sent to the server, where dashboard widgets show the current state of the environment. At the same time, local logic on the Rugged Board evaluates the temperature threshold and decides whether the buzzer should be activated.

This architecture offers both edge responsiveness and server-side observability. The local alarm works even if the user is not watching the dashboard, while the remote dashboard allows centralized monitoring and alarm history.

8. Hardware Description

The hardware design uses a Rugged Board as the central controller because it supports Embedded Linux and exposes interfaces suitable for connecting sensors and actuators. The board is responsible for I2C communication, GPIO-based alarm control, network connectivity, and execution of the monitoring application.

The BME680 sensor module provides temperature, humidity, pressure, and gas resistance measurements. Its compact design and I2C support simplify hardware integration. The SFM-27 buzzer acts as the local alert device and is controlled through one of the boards GPIO pins.

A stable power system is important for reliable operation. The project uses a 5V input and a DC-DC conversion stage to derive the 3.3V rail needed by the sensor and logic-level connections. The network path is used for communication with the ThingsBoard server.

Component	Purpose	Remarks
Rugged Board	Embedded Linux platform	Handles I2C, GPIO, and telemetry tasks
BME680 Sensor	Environmental sensing	Measures temperature, humidity, pressure, gas resistance
SFM-27 Buzzer	Audible alert output	Activated during over-temperature condition
Power Supply	Provides required operating voltages	Includes 5V input and 3.3V regulation
Ethernet / Network	Server connectivity	Used for ThingsBoard communication
On-board LED	Optional visual indication	Useful for debug or alarm indication

9. Software Description

The software environment is centered around Embedded Linux, which provides hardware abstraction, networking support, system interfaces, and development flexibility. The embedded application was developed using C and compiled using a GCC cross-compilation workflow suitable for the target platform.

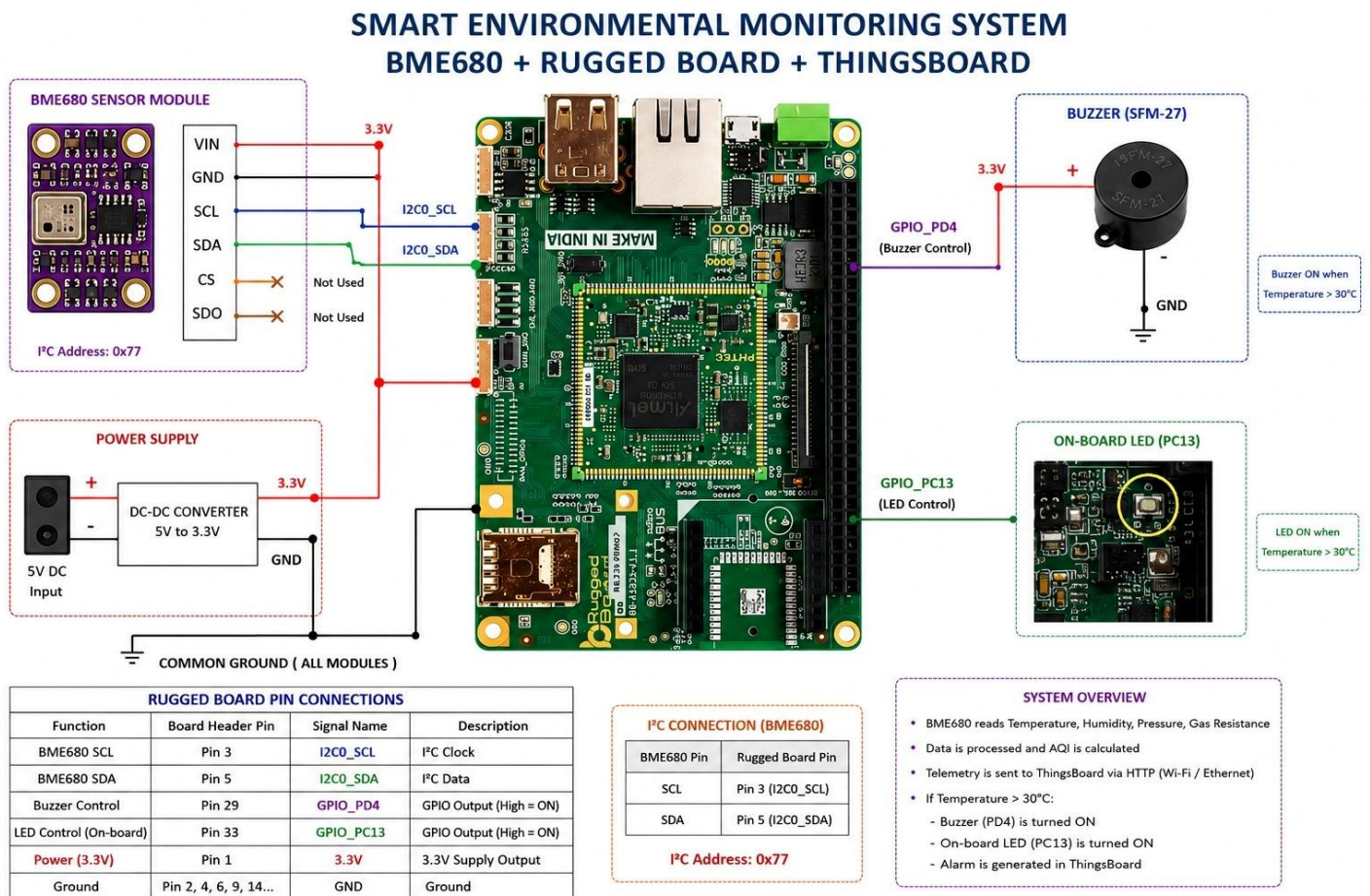
The Bosch BME68x API was used to simplify communication with the sensor and to access calibrated measurement outputs. Linux I2C support was used to verify device presence and to support the communication path required by the application.

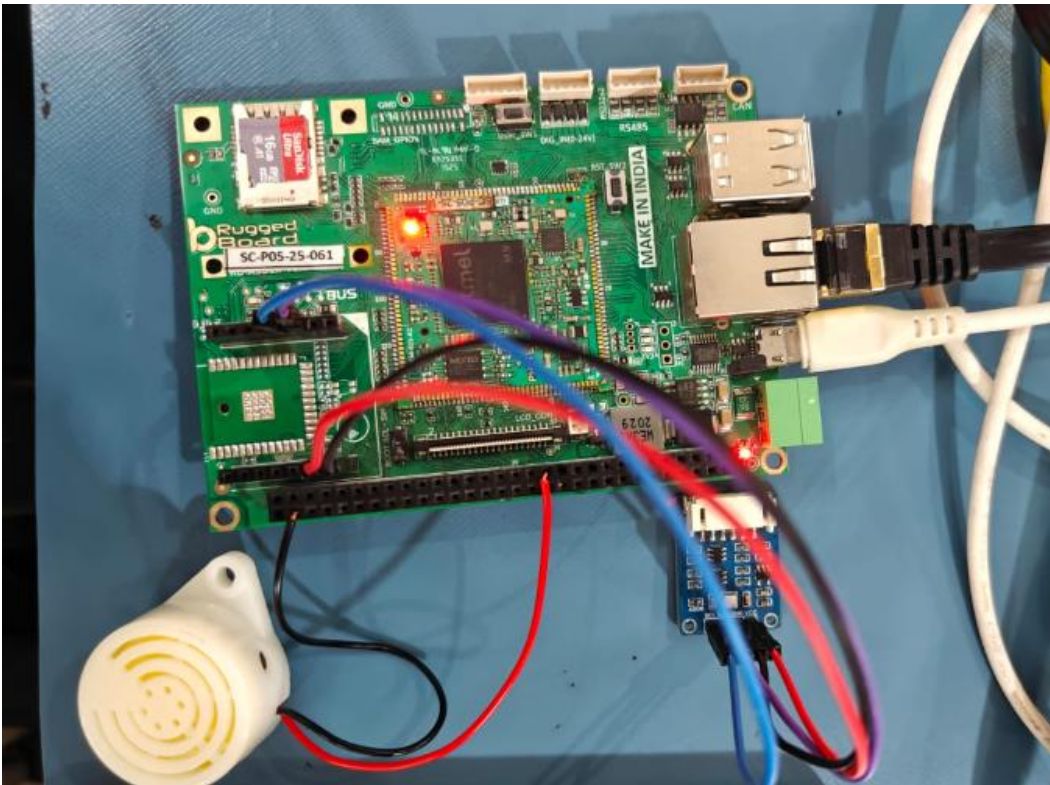
ThingsBoard Community Edition was used on the server side to receive telemetry, display dashboard widgets, and create remote alarms through rules and conditions. Together, these software elements form the complete end-to-end system.

Software / Tool	Purpose
Embedded Linux	Target operating system on the Rugged Board
GCC Cross Compiler	Builds application for the board architecture
Bosch BME68x API	Sensor interfacing support
Linux I2C Tools	Bus probing and debugging
ThingsBoard CE	Dashboard visualization and alarms
HTTP	Telemetry communication protocol
Git / GitHub	Version control and source management

10. Circuit Diagram and Wiring Details

The following figure represents the final wiring used in the project. It combines the BME680 sensor, the Rugged Board, the buzzer, the power section, and the optional on-board LED into one clear system-level diagram.





The BME680 module uses four essential connections in the selected mode: VIN, GND, SCL, and SDA. These correspond to the 3.3V supply, common ground, I2C clock, and I2C data lines on the Rugged Board.

The CS and SDO pins are shown as unused because the module is configured for I2C mode. The selected address shown in the diagram is 0x77, which should match the software configuration used during communication.

The buzzer receives power from the 3.3V line and is controlled through GPIO_PD4. The diagram also illustrates that the on-board LED connected to GPIO_PC13 can provide a secondary visual alert when the temperature exceeds the threshold.

Function	Board Pin / Signal	Connected Device Pin	Description
Power	3.3V	BME680 VIN	Sensor supply
Ground	GND	BME680 GND	Common ground
I2C Clock	I2C0_SCL / Pin 3	BME680 SCL	I2C serial clock line

I2C Data	I2C0_SDA / Pin 5	BME680 SDA	I2C serial data line
Buzzer Control	GPIO_PD4 / Pin 29	SFM-27	Alarm control output
LED Control	GPIO_PC13 / Pin 33	On-board LED	Optional visual indication

11. Sensor Interfacing Methodology

The sensor interface was implemented using the I2C communication protocol. This protocol uses two signal lines, serial clock and serial data, along with power and ground. The BME680 module was configured to operate in I2C mode, and the selected address shown in the design is 0x77.

Before running the full application, the sensor connection was verified using Linux I2C tools to ensure that the board could detect the device on the bus. This step is important because it confirms correct wiring, bus availability, and basic electrical integrity.

Once communication was confirmed, the application initialized the sensor, set the operating mode, and triggered periodic measurements. The resulting values were then parsed and used by the rest of the software pipeline.

12. Data Acquisition Flow

Data acquisition begins when the embedded application starts and configures the BME680 interface. The application then enters a loop in which it periodically requests fresh measurements from the sensor and reads the output registers or processed data structure returned by the API.

The acquired values are stored in program variables and can then be converted into telemetry-ready formats. Because multiple environmental parameters are read from one sensor, the application gains a compact snapshot of ambient conditions with each cycle.

This looped acquisition model makes the system suitable for real-time monitoring. The sampling interval can be adjusted according to network load, application responsiveness, and power requirements.

13. AQI Estimation Method

The BME680 does not directly provide an official AQI output because it does not measure particulate matter concentrations such as PM2.5 and PM10. However, its gas resistance value can still be used to build a relative indoor air quality indicator that reflects changes in air composition or ventilation conditions.

In this project, a custom AQI-style estimate was produced using gas resistance values. The intention was not to claim regulatory-grade AQI but to create a trend-friendly environmental indicator that could be visualized on the dashboard.

This approach is useful in internship and prototype settings because it helps demonstrate how raw sensor values can be transformed into more meaningful application-level information.

Estimated AQI Range	Category
0-50	Good
51-100	Moderate
101-150	Unhealthy for Sensitive Groups
151-200	Unhealthy
201-300	Very Unhealthy
301-500	Hazardous

14. Telemetry Communication using HTTP

Telemetry communication between the Rugged Board and ThingsBoard was implemented using the HTTP protocol. HTTP was selected because it is easy to test, easy to debug, and straightforward to integrate in embedded applications that already have network connectivity.

For every measurement cycle, the application forms a payload containing temperature, humidity, pressure, estimated AQI, and alarm state. This payload is then transmitted to the ThingsBoard server using the correct device endpoint and authentication mechanism configured for the project.

Although MQTT may be more efficient in larger deployments, HTTP is a good choice for a working prototype because it simplifies early integration and helps clearly demonstrate the full telemetry path.

15. ThingsBoard Dashboard Design

The ThingsBoard dashboard was designed to provide a simple and readable representation of the system state. Widgets were configured to show temperature, humidity, pressure, AQI, and alarm condition so that users can quickly understand the environment without needing to inspect raw data.

The dashboard acts as the primary remote user interface of the system. By collecting the incoming telemetry into visual widgets, it transforms the project from a low-level sensor test into a complete IoT application.

The platform also supports alarm panels and historical telemetry, which creates opportunities for future expansion into logging, analytics, and predictive monitoring.

16. Temperature Alarm Design

The local alarm mechanism was implemented using an SFM-27 buzzer connected to GPIO_PD4 of the Rugged Board. The software compares the latest temperature reading against a threshold of 30°C and decides whether the buzzer should be on or off.

This direct control path is important because local alarms must work immediately and should not depend on network latency or server availability. The use of a GPIO-driven buzzer provides a simple but effective safety response.

In addition to the buzzer, the provided system diagram also includes an optional on-board LED on GPIO_PC13. This can be used for visual status indication or debugging during development.

Alarm Parameter	Value
Temperature Threshold	30°C
Local Alarm Device	SFM-27 Buzzer
GPIO Control Pin	GPIO_PD4
Alarm Telemetry Field	alarm
Remote Alarm Platform	ThingsBoard
Severity	Critical

17. Working Principle of the System

The project starts with hardware initialization and sensor setup. Once the system is powered, the Rugged Board boots Embedded Linux, loads the required interfaces, and starts the monitoring application.

The application acquires environmental data from the sensor, computes the estimated AQI value, packages telemetry, and transmits the data to ThingsBoard. The dashboard receives the data and updates the displayed values in real time.

At the same time, the application checks the temperature threshold. When the threshold is exceeded, the buzzer is activated locally and the alarm field is sent to the dashboard so that a remote alarm becomes visible. Once conditions normalize, the system returns to its non-alarm state.

1. Power is applied and the Rugged Board boots into Embedded Linux.
2. The application initializes I2C communication and configures the BME680 sensor.
3. The sensor acquires temperature, humidity, pressure, and gas resistance values.
4. The application computes the estimated AQI value from gas resistance behavior.
5. A telemetry payload is created and sent to ThingsBoard over HTTP.
6. The dashboard updates the widgets with the latest values.
7. The software compares temperature with the threshold of 30°C.
8. If temperature is above the threshold, the buzzer is turned on using GPIO_PD4.
9. Alarm telemetry is transmitted to ThingsBoard to generate a remote alarm.

10. When the temperature returns below the threshold, the buzzer is turned off and the alarm state can be cleared.

18. Implementation Approach

The implementation was carried out in phases so that each subsystem could be tested independently before full integration. First, the hardware connections were verified. Next, the sensor communication path was confirmed through the Linux I2C subsystem.

After sensor verification, the embedded application was developed to read values and print or store the results. The next phase added HTTP telemetry transmission. Finally, alarm logic and ThingsBoard dashboard configuration were integrated into the same system.

This phased approach reduced debugging complexity and made it easier to isolate problems related to wiring, software logic, or network configuration.

19. Testing Procedure

Testing was performed progressively. Hardware wiring was checked first, followed by I2C device detection tests. Once the sensor was visible on the bus, the application was tested for initialization success and repeated measurement consistency.

Network communication tests were then performed by transmitting sample payloads and confirming reception on ThingsBoard. Dashboard widgets were checked for correct field mapping and live updates. Finally, the alarm threshold was tested by verifying local buzzer operation and remote alarm creation.

Such staged testing is essential in embedded system development because failures can come from many sources, including power issues, interface misconfiguration, incorrect payload structure, or dashboard-side mapping errors.

20. Results and Observations

The project achieved all major functional goals. The Rugged Board successfully communicated with the BME680 sensor, acquired environmental parameters, and published telemetry to the server. The ThingsBoard dashboard displayed the data in real time, and the local buzzer responded correctly when the temperature threshold was crossed.

The system also demonstrated stable integration across hardware and software layers. The combined behavior of sensing, processing, telemetry transmission, and alarm handling showed that the architecture is practical for compact IoT monitoring applications.

The results confirm that even a relatively simple prototype can provide meaningful monitoring capability when local actuation and remote visualization are designed together.

- Reliable communication was established between the Rugged Board and the BME680 sensor.
- Temperature, humidity, pressure, and gas resistance were acquired successfully.
- Telemetry transmission to ThingsBoard through HTTP worked correctly.
- Dashboard widgets displayed the measured values in real time.
- The estimated AQI logic provided a useful trend-based air quality indication.
- The local buzzer activated correctly when temperature exceeded 30°C.
- ThingsBoard generated remote alarm events successfully.
- The overall system remained stable during repeated testing.

21. Advantages of the Proposed System

The proposed system provides multi-parameter environmental sensing in a compact and practical form. It combines temperature, humidity, pressure, and gas-resistance-based air quality indication within one monitoring node.

It offers both local and remote alerting, which improves reliability and user awareness. It also demonstrates how Embedded Linux can be used effectively for industrial IoT edge applications where both hardware control and network communication are required.

Because the design is modular, it can be extended easily with additional sensors, alternative communication protocols, or improved analytics.

22. Challenges Faced

During development, one of the main challenges was ensuring reliable end-to-end integration across hardware and software domains. It was necessary to verify power integrity, bus wiring, device addressing, and correct use of the sensor API before telemetry could be trusted.

Another challenge was representing air quality in a technically honest way. Since the BME680 does not produce official AQI by itself, the report and dashboard had to describe the value as an estimated or relative indicator rather than an exact particulate-based AQI.

Configuration on the ThingsBoard side also required careful matching of telemetry field names, dashboard widgets, and alarm conditions. Small inconsistencies in naming or payload structure can prevent proper visualization or alarm generation.

23. Skills Acquired

The project strengthened several practical engineering skills, including Embedded Linux usage, cross-compilation workflows, Linux terminal debugging, I2C sensor interfacing, and GPIO control. It also improved understanding of telemetry protocols and dashboard-based monitoring.

On the software side, it provided experience in embedded C application development, data formatting, and interaction with REST-style endpoints. On the system side, it provided hands-on exposure to how edge devices integrate with IoT platforms.

Equally important, the project improved troubleshooting discipline by reinforcing the value of incremental verification and end-to-end testing.

- Embedded Linux development
- Cross-compilation workflow
- I2C communication and debugging
- Sensor interfacing using APIs
- GPIO programming and control logic
- Embedded C application development

- HTTP telemetry integration
- ThingsBoard dashboard configuration
- Alarm rule setup and validation
- System-level troubleshooting and iterative debugging

24. Limitations

The implemented system is a prototype and therefore has some limitations. The air quality value is only an estimated indication based on gas resistance and should not be interpreted as an official AQI. For regulatory or health-critical applications, dedicated particulate sensors would be required.

The current buzzer control method is sufficient for a prototype, but production hardware may require a transistor driver or dedicated interface stage depending on the buzzer current demand. The project also does not yet include long-term storage analysis, field enclosure design, or industrial certification considerations.

These limitations do not reduce the educational and demonstration value of the project, but they do define the boundary between a prototype and a deployable product.

25. Future Enhancements

The project can be extended in several directions. One improvement is the integration of the Bosch BSEC library for better indoor air quality estimation. Another is the addition of PM2.5 and PM10 sensors so that a more standard AQI can be calculated.

On the communication side, MQTT could replace HTTP for more efficient and scalable telemetry transmission. Notification features such as email, SMS, or mobile push alerts can also improve usability in real deployments.

Further development may include local data logging, historical analytics, calibration procedures, multi-node networking, and an industrial-grade enclosure for harsh environments.

26. Conclusion

This project successfully demonstrates a complete embedded environmental monitoring and temperature alarm solution using the BME680 sensor, Rugged Board, and ThingsBoard platform. It combines sensing, embedded processing, communication, local actuation, and remote visualization into one coherent Industrial IoT workflow.

The completed system meets the core objectives of real-time monitoring, dashboard integration, and alarm generation. It also provides strong evidence of practical learning in embedded systems and IoT during the internship period.

The project can serve as a strong foundation for future work in smart monitoring, factory sensing, or environmental surveillance systems where reliability, visibility, and timely alerts are important.

27. References

Bosch BME680 Sensor Datasheet.
Bosch BME68x Sensor API Documentation.
Embedded Linux Documentation.
ThingsBoard Community Edition Documentation.
Linux I2C Subsystem Documentation.
GCC Cross Compilation Documentation.